IT305 · COMPUTER NETWORKS · LAB 6

Fault-Tolerant Topic-Based File Distribution Service

Final Report — Implementation and Performance Analysis

Lab Groups 88 & 92 (Project Group PG10)

Name	Roll number	Lab group	Contribution
Meet Virugama	202401243	88	Part I server in both single- and multi-threaded modes; the shared topic table and start-up file/CRC caching in common/.
Madhav Thesiya	202401226	88	Group lead: overall architecture and wire-protocol design, integration across all four cases, the measurement harness and this report.
Krish Vamja	202401239	88	Part II Case 1: server-side fault injection, the reconnecting client with bounded backoff, and the useful/redundant byte accounting.
Pritkumar Zania	202401495	92	Part II Case 2 server side: the session table, resume validation, and the min(client, sent) clamp that bounds retransmission.
Om Sutariya	202401438	92	Part II Case 2 client side: atomic checkpoint persistence, the resume path, and CRC-32 reconstruction across a resume.
Shlok Ukani	202401204	92	Case 2 Enhanced: K-way work partitioning, the epoll connection hand-off keeping one thread per client, and the test and sanitizer suites.

Socket programming project — Part I and Part II (Case 1, Case 2, Case 2 Enhanced)

Contents

1. Introduction and scope
2. Build, layout and command-line interface
3. Part I — basic topic-based file distribution
    1. Design
    2. Implementation
    3. Results
4. Part II Case 1 — no session management
    1. Design
    2. Implementation
    3. Results
5. Part II Case 2 — with session management
    1. Design
    2. Implementation
    3. Results
6. Part II Case 2 Enhanced — parallel connections
    1. Design
    2. Implementation
    3. Results
7. Fault injection
8. Experimental method
9. Performance analysis and comparison
10. Part I on three machines
    1. Threading mode against client count
    2. The same experiment in both environments
    3. Parallel connections, measured properly
11. Deviations from the design document
12. Correctness and verification
13. Summary of observations
14. Limitations and future work

1. Introduction and scope

This report documents a topic-based file distribution service written in C11 over TCP sockets, together with the measurements taken from it. A server exposes each sub-directory of a *topics root* as a **topic**; a client names one topic and receives every file under it, writing the files out and verifying each against a CRC-32 carried on the wire.

The work is delivered in four executable pairs, one per directory, each building to exactly two binaries named `server` and `client`:

- **Part I** — the base transfer, with the server available in single-threaded and multi-threaded modes, to measure how concurrency affects aggregate transfer rate.
- **Part II Case 1** — server-side probabilistic fault injection with *no* retained state: a dropped connection costs the whole topic, which must be fetched again from the beginning.
- **Part II Case 2** — the same fault injection with session management: client and server keep enough state that a reconnect resumes from the last checkpointed byte.
- **Part II Case 2 Enhanced** — session management plus *K* parallel TCP connections per client, multiplexed inside a single server thread with `epoll`.

All four share one library in `common/` holding the wire protocol, the topic table, CRC-32, logging, fault injection and argument parsing. The complete wire format — every message type, field layout, flow and invariant — is specified in `PROTOCOL.md`, which is the authoritative reference; this report describes the design at the level needed to interpret the measurements, and does not restate the byte layouts.

Performance was measured in two campaigns. The main matrix (Section 9) is **340 runs** on a single host over loopback, of which **340** completed with `status=ok`; across them **2,147,270 files** were delivered and CRC-verified with **0 mismatches**, totalling **28.9 GB** of useful data and 36.5 GB on the wire. Part I and the Enhanced case were then repeated on **three separate machines** over a wireless network (Section 10): a further **52 runs**, again all `status=ok`, with 0 CRC failures.

Keeping both was necessary rather than redundant. Byte counts are a property of the protocol and are identical in either environment, so the loopback matrix is the right place to measure them at scale. Throughput is not, and Section 10 shows two conclusions that loopback alone would have got wrong.

2. Build, layout and command-line interface

Every directory builds independently with a plain `make`, using `-std=c11 -Wall -Wextra -Werror -O2`, so any warning is a build failure. There are no third-party libraries and no install step; the shared library is compiled by relative path.

```
FinalSubmission_Group_088_092/
├── FinalReport_Group_088_092.pdf   this report
├── README.md                       build / run / test / reproduce
├── PROTOCOL.md                     wire-format specification
├── Part1/                          server.c  client.c  Makefile
├── Part2_Case1/                    + fault injection
├── Part2_Case2/                    + session.[ch]  checkpoint.[ch]
├── Part2_Case2_Enhanced/           + workplan.[ch]
├── common/                         shared library (9 modules)
├── tests/                          unit + 4 functional suites + sanitizers
├── bench/                          measurement harness and plotting
├── report/                         sources for this PDF
└── Results/                        bench_runs.csv, SUMMARY.md, graphs
```

The mandatory invocations required for automated grading are honoured exactly. Part I accepts an optional third positional argument selecting the threading mode, and every other tunable is an optional trailing `--flag` with a compiled-in default, so the required forms always work unmodified:

```
./server <port> <topics_root_dir>                      # Part 1 (defaults to multi)
./server <port> <topics_root_dir> [single|multi]       # Part 1, mode selected
./server <port> <topics_root_dir> <failure_prob>      # Part 2, all cases
./client <server_ip> <server_port> <topic> <output_dir>
./client <server_ip> <server_port> <topic> <output_dir> [K]   # Enhanced, K defaults to 4
```

Nothing hard-codes a path, port or address; the topic table is built by scanning `topics_root_dir` at start-up, so a reference dataset supplied by a grading script works without recompilation. Each client prints one machine-readable `RESULT` line on completion carrying useful bytes, redundant bytes, bytes on the wire, elapsed time, goodput, overhead, reconnects and CRC counts — this line is the raw material for every table in Section 9.

3. Part I — basic topic-based file distribution

3.1 Design

The server keeps a read-only **topic table**, built once before the listening socket opens: each immediate sub-directory of the topics root becomes a topic key, and its sorted file list, per-file sizes and per-file CRC-32 are cached alongside it. Caching does two jobs. It removes a directory scan and a checksum pass from every request, and — more importantly — it fixes one canonical file order that client and server agree on with no negotiation, which is what later lets a numeric `file_idx` identify a file during a resume. Because the table is never mutated after start-up, lookups need no lock even in the multi-threaded server.

Every message in every part shares one framing rule: a fixed 6-byte header (`msg_type`, `flags`, `payload_len`) followed by a payload whose length the header declares. One `read_message()` / `write_message()` pair therefore serves all four executables. Part I uses six message types: `REQ_TOPIC`, `RESP_META`, `RESP_FILE_INFO`, `RESP_FILE_CHUNK`, `RESP_DONE` and `RESP_ERROR`.

```
Client                                  Server
  |-- REQ_TOPIC("gallina") ------------->|  look up in the topic table
  |<------------ RESP_META(3098) --------|  (absent -> RESP_ERROR, close)
  |      for each file, in cached sorted order:
  |<----- RESP_FILE_INFO(name,size,crc32)|
  |<----- RESP_FILE_CHUNK x ceil(size/16384)
  |<------------ RESP_DONE --------------|  client verifies every CRC-32
```

`RESP_FILE_CHUNK` deliberately carries no offset field: within one file chunks are strictly sequential, so the receiver's running offset is implicit and per-chunk overhead is just the 6-byte header per 16 KiB, or 0.037 %. A zero-length file is announced with `file_size = 0` and followed by no chunks at all; the synthetic test set contains such a file precisely to keep that path exercised.

3.2 Implementation

Both threading modes live in one `server.c` and share the entire request path; they differ only in what happens after `accept()`.

Single-threaded
: `handle_request()` is called inline on the accepting thread and runs to completion, so a second client waits in the kernel's listen backlog until the first is fully served. This is the baseline the comparison needs.

Multi-threaded
: Each accepted connection is handed to a new detached worker thread and the accept loop immediately returns to `accept()`. Because the topic table is read-only, workers share it with no synchronisation at all.

Beyond the minimum, the implementation adds: structured timestamped logging to stderr or a file; a `SIGINT` / `SIGTERM` handler that stops accepting and lets in-flight workers finish; per-file CRC-32 verification on the client; rejection of any topic or file name containing `..`, a leading `/`, or (for file names) any `/`; and `MSG_NOSIGNAL` on every send with `SIGPIPE` ignored process-wide, so a vanished peer yields `EPIPE` rather than killing the process.

3.3 Results

Aggregate transfer rate is total useful bytes delivered across all clients divided by the makespan — the wall-clock time from launching the first client until the last one finishes. Topic `cane`, no faults, with the mean over

repeats and ±1 standard deviation.

clients	single: time (s)	single: Mbit/s	multi: time (s)	multi: Mbit/s	multi / single
1	0.73 ±0.22	735 ±253	0.33 ±0.05	1521 ±226	2.07×
2	0.81 ±0.26	1346 ±462	0.40 ±0.08	2532 ±391	1.88×
4	2.43 ±0.44	823 ±133	1.04 ±1.18	3087 ±1298	3.75×
8	4.01 ±1.12	1056 ±317	1.45 ±0.59	3361 ±1796	3.18×



Figure 1. Aggregate transfer rate against the number of concurrent clients, for the single-threaded and multi-threaded servers.

The multi-threaded server is ahead at every client count, and the gap widens with concurrency: at 8 clients it delivers **3.18×** the aggregate rate of the single-threaded server. The shape is what the two designs predict. The single-threaded server serialises transfers, so adding clients cannot add throughput — its aggregate rate rises only 1.44× from 1 to 8 clients, and that rise comes from overlapping one client's disk reads with another's socket writes rather than from any real parallelism. The multi-threaded server scales 2.21× over the same range, limited by cores and by memory bandwidth on loopback rather than by the protocol.

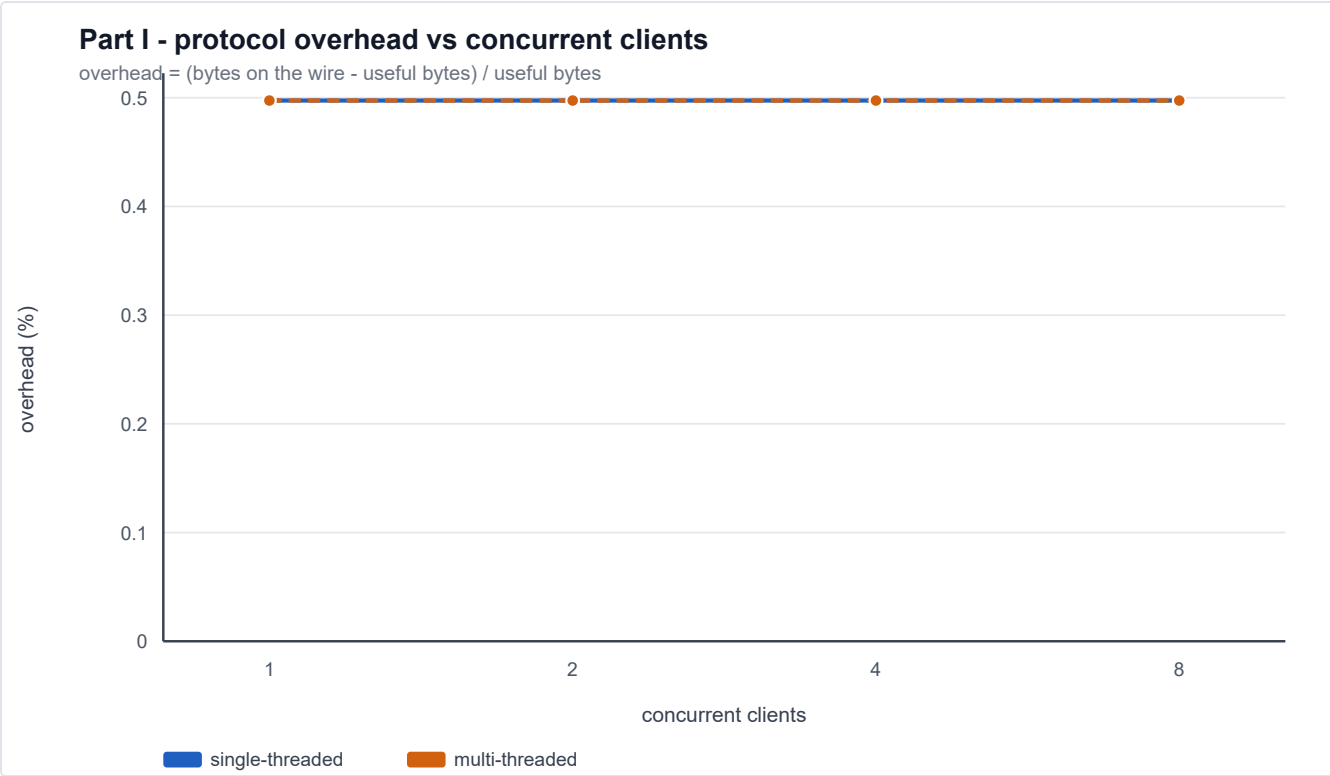


Figure 2. Protocol overhead against concurrent clients. Overhead is (bytes on the wire − useful bytes) / useful bytes.

Overhead is flat and far below 1 % in both modes, and it is independent of concurrency, exactly as the framing predicts: the only non-payload bytes are the 6-byte chunk headers plus one `RESP_FILE_INFO` per file, neither of which depends on how many clients are connected.

Transfer rate also varies with the shape of a topic, not only its size:

topic	files	MB	chunks	time (s)	Mbit/s	overhead %
scoiattolo	1,862	24.2	1551	0.18 ±0.06	1202 ±306	0.458
cane	4,863	57.9	3705	0.57 ±0.15	923 ±279	0.497
gatto	1,668	144.1	9226	0.91 ±0.72	1771 ±661	0.088

Overhead falls as mean file size rises, because the per-file `RESP_FILE_INFO` cost is amortised over more payload. This is why `gatto` (mean file 88 KB) shows markedly lower overhead than `cane` (mean file 12 KB) despite carrying far more data.

4. Part II Case 1 — no session management

4.1 Design

Case 1 reuses the Part I message set unchanged. The only additions are server-side fault injection (Section 7) and, on the client, detection of a mid-transfer disconnection followed by a fresh connection and a fresh `REQ_TOPIC`. Neither side retains any record of the previous attempt, so the server treats the reconnect as an entirely new transfer and streams the topic again from `RESP_META` onward.

4.2 Implementation

The client retries automatically with no manual step. Backoff doubles from 10 ms to a 500 ms cap and the attempt count is bounded by `--retries` (default 500), so a pathological fault rate terminates with a diagnostic instead of looping forever. Every byte received before a fault is discarded, and the counters separate **useful** bytes (those present in the final output) from **redundant** bytes (those that had to be sent more than once), which is what makes the Case 1 / Case 2 comparison quantitative rather than anecdotal.

> **The cost model.** Because a fault discards the whole topic, a pass over C chunks succeeds with probability $(1-p)^C$, so the expected number of passes is $(1-p)^{-C}$ and expected bytes on the wire are that multiple of the topic size. This grows *explosively* in C, and it is the single most important fact about Case 1.

4.3 Results

Measured on topic `scoiattolo` (C = 1551 chunks). The columns are the five quantities the comparison calls for: **useful MB** is the data actually delivered to the client's disk (constant, since the same topic is fetched every time); **total wire MB** is everything the client received, headers included; **redundant MB** is the part of that which had to be sent more than once; then **completion time** and **throughput against overhead**. "Expected faults/pass" is $C \times p$, and "× topic" expresses redundant bytes as a multiple of the topic size.

p	exp. faults/pass	useful MB	total wire MB	redundant MB	× topic	time (s)	Mbit/s	overhead %	reconnects
0	0.00	24.2	24.3	0.0	0.00×	0.15 ±0.04	1404 ±314	0.46	0.0
0.0002	0.31	24.2	30.6	6.3	0.26×	0.28 ±0.23	1106 ±543	26.40	0.6
0.0005	0.78	24.2	59.3	34.8	1.44×	1.50 ±1.81	579 ±509	144.64	3.1
0.001	1.55	24.2	116.8	92.1	3.80×	6.73 ±5.26	93 ±120	382.13	12.9
0.002	3.10	24.2	498.4	471.8	19.47×	52.01 ±43.92	16 ±31	1956.64	96.0

The prediction holds. Redundant traffic is negligible at p=0, and by p=0.002 the client moves **471.8 MB** of redundant data to deliver a 24 MB topic. Completion time degrades in step. The spread between repeats is very large at high p, which is not measurement noise but the geometric distribution of the number of passes: one lucky run completes first time, another needs twenty.

5. Part II Case 2 — with session management

5.1 Design

Case 2 adds an 8-byte, server-generated `session_id` and five message types around it: `REQ_SESSION_START`, `RESP_SESSION_ID`, `REQ_SESSION_RESUME`, `RESP_RESUME_ACK`, `RESP_CHECKPOINT` and `REQ_ACK`. A position is a `(file_idx, byte_offset)` pair meaning "every file below `file_idx` is complete on the client's disk, and the first `byte_offset` bytes of `file_idx` are on the client's disk".

That definition is deliberately precise. A client that has just finished file 5 is at `(6, 0)`, never `(5, size_of_5)` — otherwise a zero-length file at the resume point would be indistinguishable from one still to come.

```
... streaming file 46 ...  [fault injected] -> server closes the socket
client keeps its on-disk checkpoint (file 46, offset 94208)
|== new TCP connection ==                   |
|-- REQ_SESSION_RESUME(id, 46, 94208) ----->|  validate, then clamp
|<-- RESP_RESUME_ACK(id, 46, 81920) --------|
|<-- RESP_FILE_INFO(file 46) [FLAG_RESUMED] |  full size is announced
|<-- RESP_FILE_CHUNK from offset 81920 .....|  only the gap is re-sent
|      ... continues with files 47..N ...   |
```

5.2 Implementation

The **client** is the side that must survive a restart, so it is the client that persists state: every 64 KB of progress, or at a file boundary, it writes `output_dir/topic/.checkpoint` as `.checkpoint.tmp` and `rename()`s it into place, so a crash can never leave a torn record. The **server** holds sessions in a mutex-guarded hash table keyed by `session_id`, evicting them after 60 s idle so memory is bounded even if a client disappears.

Two details matter for correctness and are easy to get wrong:

- **The resume clamp.** The server resumes from `min(client_position, server_sent_position)` compared lexicographically, never advancing past data it has actually transmitted. It honours the client's claim because the client writes its checkpoint *before* sending the matching `REQ_ACK`, so a claim is always backed by durable state. If validation fails the server answers `start_file_idx = 0xFFFFFFFF` and the client restarts from scratch — *restart rather than guess*, because a wrongly accepted resume would silently leave a hole in a file.
- **Rebuilding the CRC.** CRC-32 is sequential, so a client resuming file *f* at offset *o* has no running state for the first *o* bytes. It re-reads them from its own output file, folds them into a fresh running CRC, then seeks and continues — which is what keeps the per-file integrity check exact across any number of resumes. A file shorter than *o* means the checkpoint is untrustworthy, and the client falls back to a fresh session.

5.3 Results

p	exp. faults/pass	useful MB	total wire MB	redundant MB	× topic	time (s)	Mbit/s	overhead %	reconnects
0	0.00	24.2	24.4	0.0	0.00×	0.86 ±0.22	260 ±95	0.62	0.0
0.0002	0.31	24.2	24.4	0.0	0.00×	1.21 ±0.69	230 ±118	0.63	0.5
0.0005	0.78	24.2	24.4	0.0	0.00×	1.05 ±0.66	236 ±75	0.66	1.7
0.001	1.55	24.2	24.4	0.0	0.00×	3.03 ±1.04	76 ±27	0.79	5.4
0.002	3.10	24.2	24.4	0.0	0.00×	3.33 ±3.10	98 ±55	0.77	8.0

> **Headline result.** At p=0.002 on `scoiattolo`, Case 1 retransmits **471.8 MB** while Case 2 retransmits **0.01 MB** — a factor of about **60395×** less redundant traffic — and completes about 15.6× faster (3.3 s against 52.0 s).

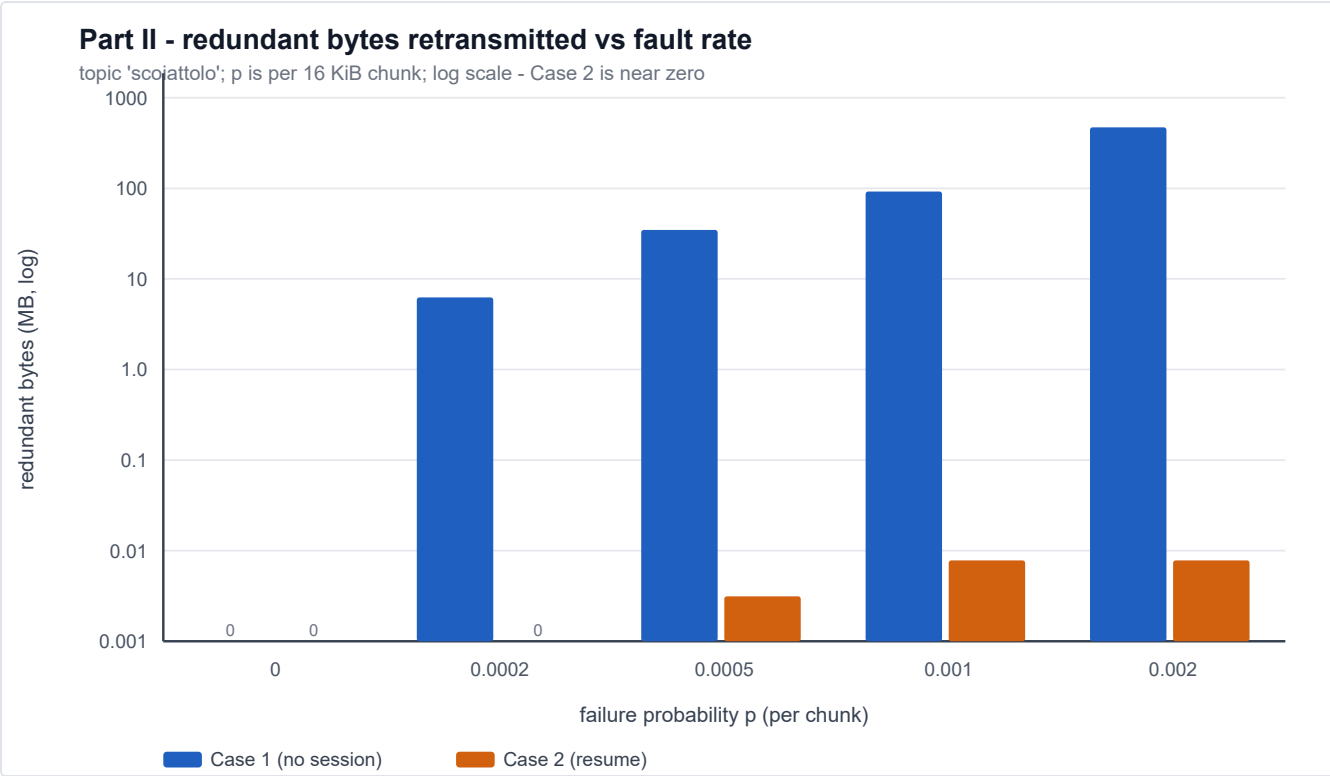


Figure 3. Redundant bytes retransmitted against fault rate, Case 1 against Case 2 (log scale). Case 2 stays flat at or near zero while Case 1 grows without bound.

Case 2's redundant traffic is bounded by construction: a fault costs at most the bytes between the last checkpoint and the drop, so it is bounded by one checkpoint interval per fault regardless of topic size or fault rate. The graph shows that directly — Case 2's curve is flat near zero on a log axis across the whole sweep, while Case 1's rises steeply.

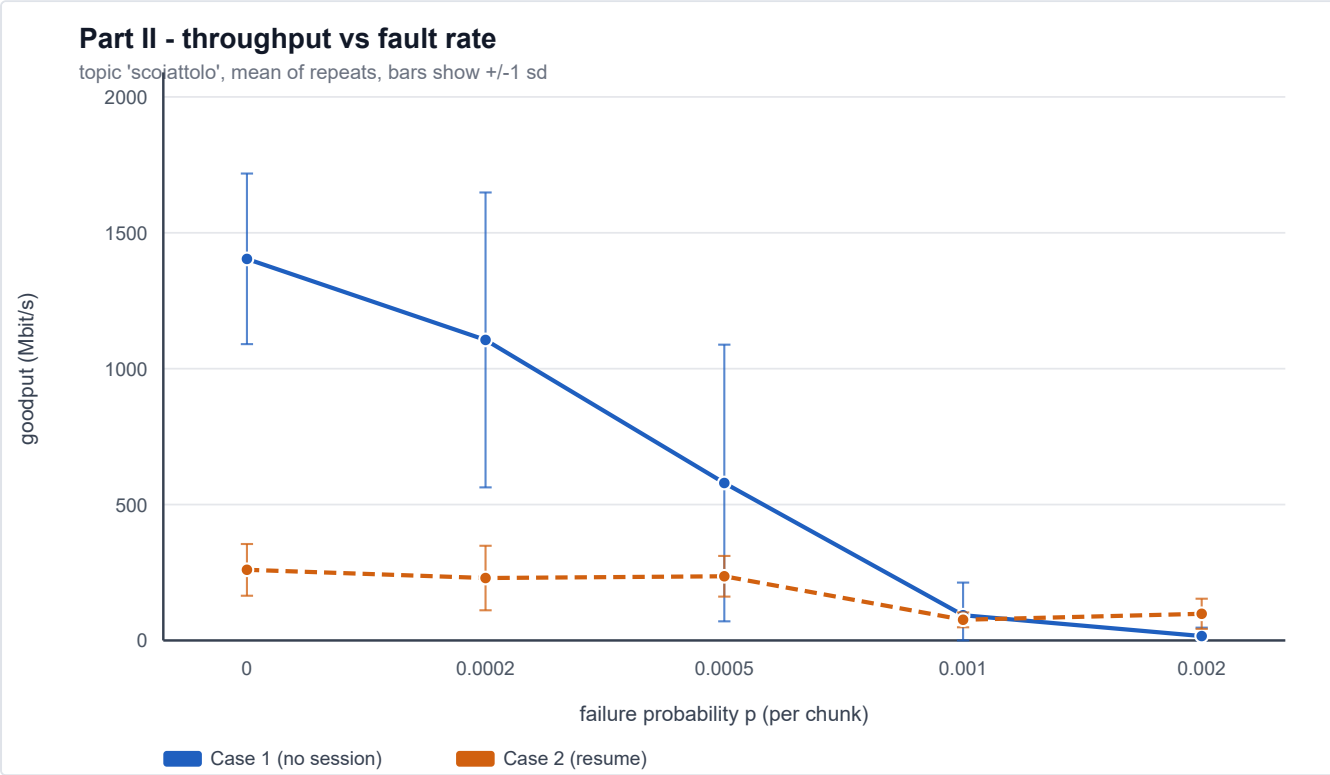


Figure 4. Goodput against fault rate for both cases.

The same comparison on a large-file topic, `gatto` (C = 9226 chunks), where restart-from-scratch is most expensive. Note the much lower p: at this chunk count even p=0.001 would need roughly ten thousand Case 1 passes, which is not a measurable experiment.

p	exp. faults/pass	useful MB	total wire MB	redundant MB	× topic	time (s)	Mbit/s	overhead %	reconnects
0.0001	0.92	144.1	300.4	156.0	1.08×	1.00 ±0.80	2208 ±1455	108.43	2.8

p	exp. faults/pass	useful MB	total wire MB	redundant MB	× topic	time (s)	Mbit/s	overhead %	reconnects
0.0001	0.92	144.1	144.3	0.0	0.00×	1.18 ±0.93	1287 ±350	0.13	1.5

Here Case 1 moves 156.0 MB of redundant data against Case 2's 0.02 MB. Completion times are much closer than the byte counts, and that discrepancy is itself the finding: on loopback, re-sending hundreds of megabytes is nearly free in time, so the time axis understates the cost of Case 1. On any real link the redundant-byte column is what would dominate.

6. Part II Case 2 Enhanced — parallel connections

6.1 Design

The Enhanced case adds byte-range requests so one client can pull a topic over K TCP connections at once (default $K = 4$). The client fetches the file list once with `REQ_FILE_LIST`, then builds an ordered list of **work items**, each a `(file_idx, range_start, range_end)` triple:

- a file below the 512 KB split threshold becomes one work item covering the whole file, assigned to connection `file_idx mod K`;
- a file at or above it is split into K contiguous ranges, range j going to connection j.

Every Enhanced transfer is therefore a range request, which keeps the server's state machine to a single case. `RESP_RANGE_CHUNK` carries an explicit 12-byte `(file_idx, abs_offset)` prefix because a receiver with K concurrent streams cannot infer position from a shared cursor; the client `pwrite()`s at the absolute offset, so the connections may finish in any order.

6.2 Implementation: one thread per client, not per connection

The assignment requires one thread serving each client. With K sockets per client the naive reading — one thread per connection — would put K server threads to work for a single client, which is exactly what the requirement rules out. The implementation keeps the mapping one-to-one at the *client* level instead:

1. The first connection carries `REQ_SESSION_START` and spawns that session's **owner thread**.
2. Every later connection from the same client arrives with `REQ_ATTACH(session_id, conn_index)`. The listener does not spawn a thread for it; it pushes the file descriptor onto the owner's hand-off queue (an `eventfd` plus a small array) and returns to `accept()`.
3. The owner runs one `epoll_wait()` loop over its eventfd and its K non-blocking sockets, serving whichever is ready and reading file data with `pread()` so the streams never share a file offset.

Server thread count therefore tracks concurrently active **clients**, not TCP connections, in every part and case. Non-blocking I/O is adopted narrowly, only where the threading constraint leaves no alternative; Part I, Case 1 and Case 2 keep simple blocking sockets, where a thread has nothing else to do while it waits.

Each connection keeps its own progress sub-record, so a fault on one connection triggers a resume for only that sub-range and the other K−1 connections keep streaming.

6.3 Results

K	time (s)	Mbit/s	vs K=1	reconnects	overhead %
1	3.65 ±2.25	196 ±109	1.00×	0.0	1.04
2	2.63 ±1.10	225 ±103	1.15×	0.0	1.04
4	1.39 ±0.34	366 ±65	1.87×	0.0	1.04
8	1.68 ±1.01	337 ±83	1.72×	0.0	1.04

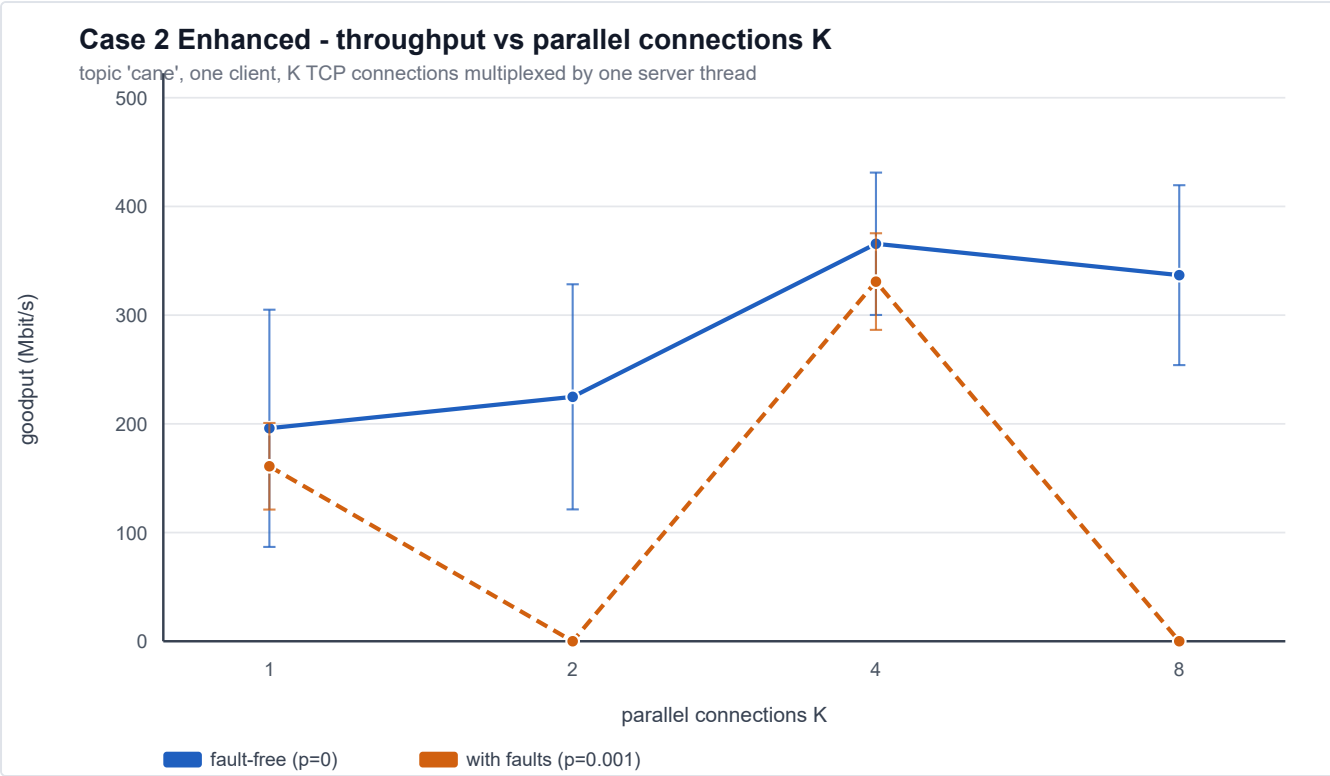


Figure 5. Enhanced-case goodput against the number of parallel connections K, with and without faults.

> **Parallel connections pay off, but sub-linearly and only up to a point.** K=4 delivers **1.87×** the goodput of a single connection; K=8 falls back to 1.72×.

The shape of that curve is worth explaining, because the usual argument for parallel connections does not apply on this test bed. Multiple streams normally help by escaping one TCP connection's congestion window or by hiding round-trip latency, and loopback has neither — yet the speedup is real. The mechanism here is different: with a single connection the session's owner thread alternates strictly between `pread()` and `send()` on one socket, so every time that socket's send buffer fills, the whole transfer stalls. With K sockets in its `epoll` set the thread almost always has some descriptor ready to write, so it stays busy; and on the client side the K receiving threads spread the `pwrite()` and CRC-32 work across cores. Both effects saturate once the single owner thread is fully occupied, which is what the regression at K=8 shows — past that point extra connections add `epoll` churn and per-range bookkeeping without adding capacity.

Under faults the gain is, if anything, slightly larger, because a fault costs only the affected connection while the other K−1 keep streaming:

K	time (s)	Mbit/s	vs K=1	reconnects	overhead %
1	3.26 ±0.98	161 ±40	1.00×	5.7	1.04
4	1.49 ±0.20	331 ±44	2.06×	5.7	1.04

Here K=4 reaches 2.06× the single-connection goodput at the same fault rate and the same number of reconnects, confirming that the per-connection resume path does not serialise the other connections.

> **These loopback figures badly understate the mechanism, and Section 10.3 shows by how much.** Parallel connections exist to hide per-flow limits and round-trip latency, neither of which loopback has, so the gain measured here comes only from keeping the owner thread busy. Repeated over 802.11 the same sweep yields **5.32× at K=8** and is still rising. Read this table as a lower bound.

7. Fault injection

The server takes the failure probability p as its third positional argument. After every emitted 16 KiB data chunk a worker draws a uniform random number and, with probability p, abandons the connection without sending `RESP_DONE`, so the client observes a mid-transfer disconnection.

> ***p* is per 16 KiB chunk** — not per transfer and not per second. A topic of C chunks therefore sees $C{\times}p$ expected faults per pass, and completes a pass untouched with probability $(1{-}p)^C$. Reading p as a per-transfer probability would make every number in Section 9 look wrong by orders of magnitude.

Evaluating per chunk rather than per file is deliberate: faults then land at arbitrary byte offsets inside a file and genuinely exercise Case 2's byte-offset resume path, instead of only ever firing on file boundaries. Each worker owns its own `rand_r()` state, so workers never contend. Per-worker streams derive from one process-wide seed that is logged at start-up and can be pinned with `--seed`, so an entire run — not merely a statistically similar one — can be replayed, which is what allows two cases to be compared under an identical fault pattern.

Chunk counts for the dataset, which is what makes a choice of p concrete:

```
scoiattolo 1551   mucca   1577   cavallo 2284   gallina 3165   elefante 3572
cane       3705   farfalla 3777  pecora  3823   ragno   4822   gatto    9226
```

Choosing p is therefore constrained by Case 1's cost model. On `scoiattolo` the expected number of Case 1 passes is 1.4 at p=0.0002, 4.7 at 0.001, 22 at 0.002 and about **2,300** at 0.005 — some 55 GB of transfer, and past the client's 500-reconnect cap. The sweep therefore stops at 0.002, and the large-file comparison on `gatto` uses 0.0001. Case 2 and the Enhanced case resume from a checkpoint and are not subject to this limit at all, which is precisely the point the comparison makes.

8. Experimental method

Measurements come from `bench/run_bench.sh`, which drives seven experiments, writes one row per run to `Results/bench_runs.csv`, and is the only source of every number in this report. `bench/summarize.py` and `bench/plot_results.py` reduce that CSV to the tables and graphs; neither re-measures anything, so the analysis can always be regenerated from the raw data.

Dataset. Animals-10: 10 topics, 26,179 files, 585.9 MB, mean file size 23 KB. Experiments use `scoiattolo` (24 MB, smallest), `cane` (58 MB, most files) and `gatto` (144 MB, largest files).

Controls. The page cache is warmed before each timed group, so the first-touch cost of reading hundreds of megabytes never lands inside a timed run. Fault seeds are pinned per repeat. Stochastic configurations are repeated ten times and reported as mean ±1 standard deviation. Each run is verified before its output is counted: a run contributes to a table only if the client reported `status=ok` and `crc_fail=0`.

8.1 Two test beds

Every experiment was run in **two** environments, and the report keeps them strictly apart.

Loopback (one host)
: The full matrix of Section 9: 340 runs on a single machine over `lo`. The loopback MTU is 65536 bytes against Ethernet's 1500, there is no physical medium and the round-trip time is microseconds, so throughput is bounded by memory bandwidth rather than by a link and lands in the Gbit/s range. These numbers compare the four implementations against *each other*; they are not link rates.

Three machines (Section 10)

52 runs with the server on one host and clients split across two others over 802.11, exactly as the assignment specifies. Peak aggregate rate observed: 178 Mbit/s.

The distinction matters because the two disagree. Byte counts — useful, redundant and total on the wire — are properties of the protocol and are identical in both. Throughput and completion time are not, and Section 10 shows that two conclusions drawn from loopback alone would have been wrong.

> **A timing caveat.** WSL2's `CLOCK_REALTIME` can step backwards when it resynchronises with the host; an early validation pass recorded a wall span of −0.42 s. The harness therefore accepts a wall-clock span only when it is positive and at least as large as the longest client-measured transfer, falling back to the latter otherwise, and records both figures in the CSV so any substitution stays visible.

9. Performance analysis and comparison

9.1 The standing cost of fault tolerance

With fault injection disabled (p=0) and one client, every implementation transfers identical bytes, so any difference is the price of the machinery each one carries. This comparison is run as its own experiment on a single topic (`scoiattolo`), because the other experiments choose whichever topic suits what they measure and their p=0 rows would otherwise sit on different workloads:

implementation	time (s)	Mbit/s	vs Part I	overhead %
Part I (multi-threaded)	0.09 ±0.01	2384 ±350	1.00×	0.458
Case 1 (no session)	0.10 ±0.03	2228 ±521	0.93×	0.458
Case 2 (session management)	0.56 ±0.08	375 ±57	0.16×	0.619
Case 2 Enhanced, K=1	0.69 ±0.10	301 ±39	0.13×	0.962
Case 2 Enhanced, K=4	0.53 ±0.04	387 ±31	0.16×	0.962

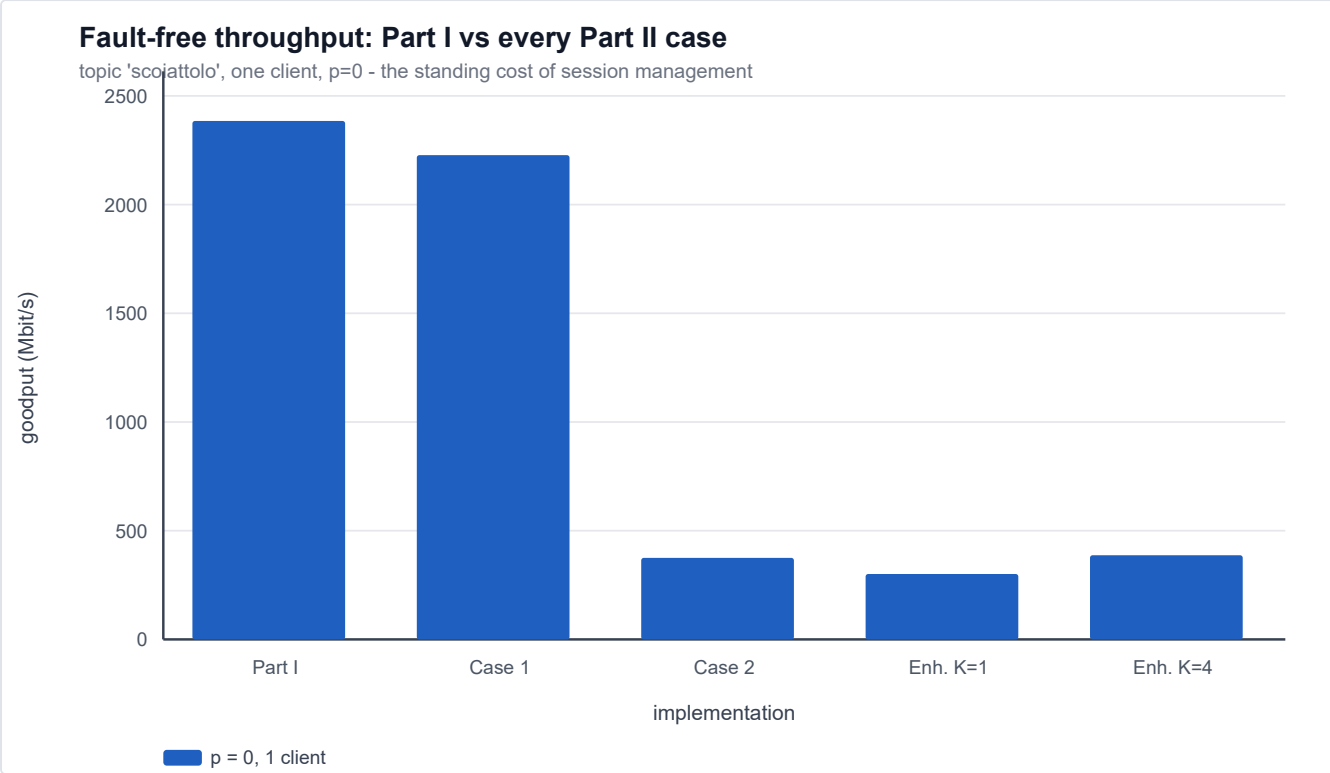


Figure 6. Fault-free goodput for Part I and each Part II case, one client. The gap is the standing cost of session management.

Case 1 costs almost nothing relative to Part I, which is expected: it is Part I plus a branch after each chunk. Case 2 is substantially slower even though no fault ever occurs, and the cost is structural rather than incidental: a checkpoint file rewritten and `rename()`d every 64 KB, a `REQ_ACK` on the same cadence, and a `RESP_CHECKPOINT` at every file boundary. On a 24 MB topic of mostly small files that is thousands of extra small synchronous operations, and on loopback — where the transfer itself is nearly free — they dominate. On a real network link the same absolute cost would be a small fraction of transfer time.

The Enhanced case at K=1 is slower still, as it must be: it adds the work-plan exchange, the 12-byte range-chunk prefix and per-range acknowledgements on top of Case 2's machinery, and a single connection cannot exploit any of the parallelism that pays for them. At K=4 it recovers that cost and edges past Case 2, which is

the case the default configuration actually ships — K defaults to 4. Its overhead column is the highest of the four, at roughly twice Part I's, and that is the measured price of the explicit `(file_idx, abs_offset)` prefix each range chunk carries.

9.2 Throughput against overhead

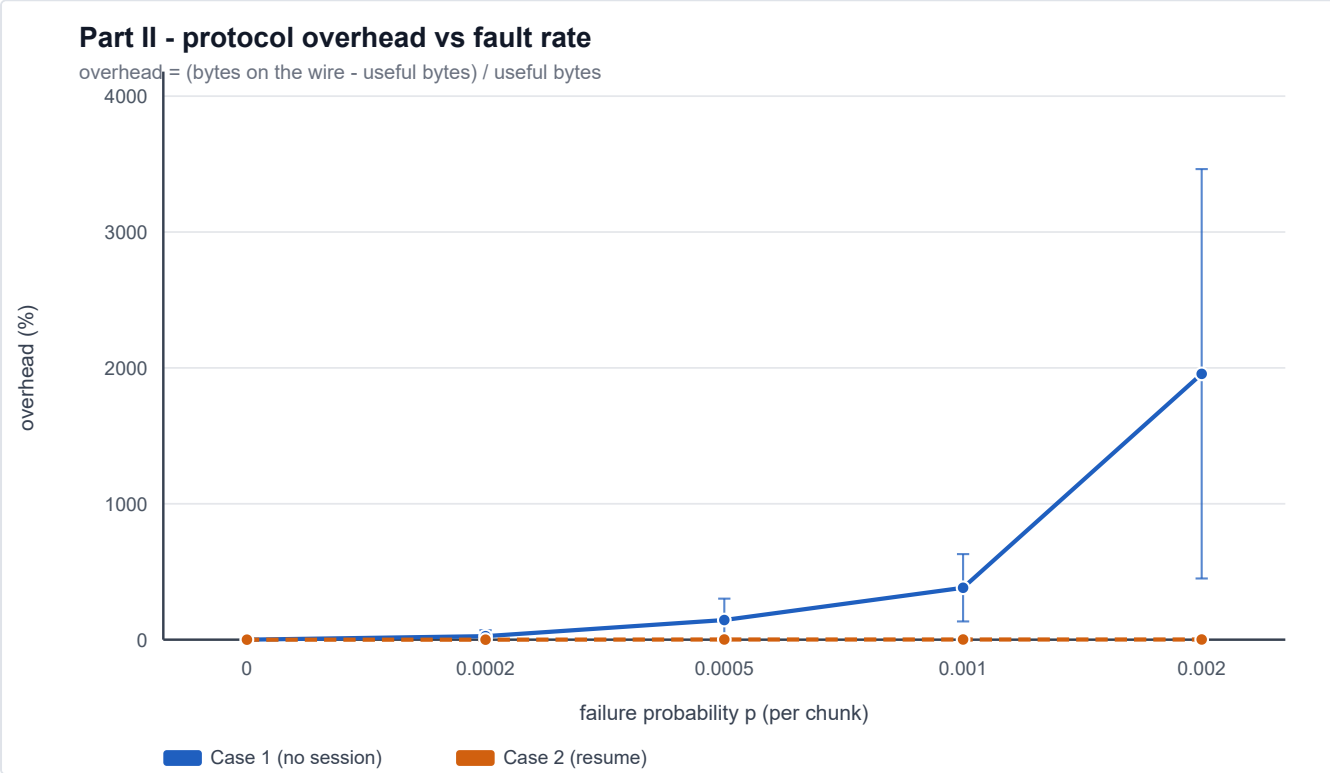


Figure 7. Protocol overhead against fault rate. Case 2 stays at the fault-free baseline; Case 1 rises with every restarted pass.

Overhead separates the two cases cleanly. Case 2's overhead is essentially constant in p, because a fault costs it one bounded gap. Case 1's rises without bound, because every restarted pass re-sends payload that was already delivered. Framing overhead itself is tiny in every configuration — well under 1 % — so at high p Case 1's overhead is almost entirely redundant payload rather than protocol headers.

9.3 Comparison against Part I

The assignment asks that Part II results be compared against Part I values. Three observations follow from the tables above.

- **Fault-free, Part II Case 1 is Part I.** Same message set, same flow, one extra branch per chunk; the measured difference is within run-to-run spread.
- **Session management trades throughput for bounded loss.** Case 2 is slower than Part I at p=0 by roughly 6.4× on this test bed, and that is the price of checkpointing. From the first injected fault onward the trade reverses decisively, and by p=0.002 Case 2 is 15.6× *faster* than Case 1 while moving 60395× less redundant data.
- **The break-even point is very low.** Case 2 overtakes Case 1 as soon as the expected number of Case 1 passes exceeds the checkpointing overhead ratio — on `scoiattolo` that happens well below p=0.0005, i.e. at under one expected fault per pass.

10. Part I on three machines

The assignment requires Part I to be measured with the server on one node and clients on two others. That experiment was run on a separate test bed:

```
server   MADHAV             10.110.170.129   Ubuntu 24.04 (WSL2, mirrored networking)
                                              Intel Core Ultra 5 125H, 18 logical cores
client   Bhanderis-Laptop    10.110.170.122   macOS
client   Meets-MacBook-Air   10.110.170.155   macOS
link     802.11 wireless, one private subnet, peak 178 Mbit/s observed
```

52 runs, all reaching `status=ok` with 0 CRC failures over 787,806 files. `Results/distributed/server_runs.csv` records one row per client connection; its `peer` field shows every connection arriving from one of exactly two addresses, which is the evidence that server and clients ran on separate machines.

Two controls were added after a first pass proved biased. The page cache is warmed once before any timed run, and the two threading modes are interleaved within each repeat rather than run as consecutive blocks. In the first pass all the single-threaded runs went first, from a cold cache, and the multi-threaded server appeared 2.13× faster at one client; with the bias removed the same figure is 1.23×. The uncorrected pass is retained as `Results/distributed_pass1/`.

10.1 Threading mode against client count

clients	single: time (s)	single: Mbit/s	multi: time (s)	multi: Mbit/s	multi / single
1	6.62 ±3.92	92 ±32	4.38 ±0.54	113 ±14	1.23×
2	8.61 ±1.65	117 ±22	7.76 ±1.14	128 ±17	1.09×
4	15.58 ±2.60	128 ±22	12.09 ±0.90	162 ±12	1.26×
8	35.34 ±4.63	112 ±16	28.77 ±2.25	136 ±11	1.21×

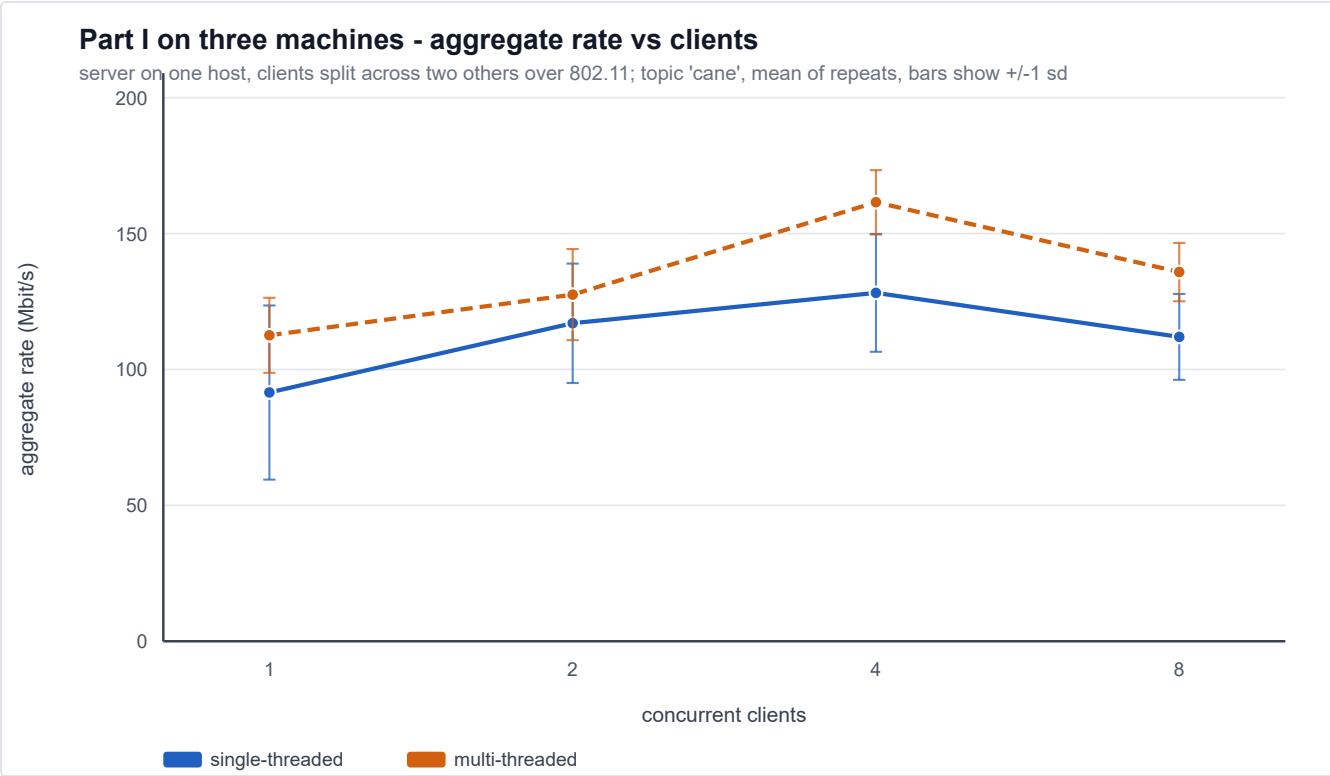


Figure 8. Part I on three machines: aggregate transfer rate against concurrent clients, for both threading modes, over 802.11.

10.2 The same experiment in both environments

clients	loopback single	loopback multi	loopback ratio	network single	network multi	network ratio
1	735	1521	2.07×	92	113	1.23×
2	1346	2532	1.88×	117	128	1.09×
4	823	3087	3.75×	128	162	1.26×
8	1056	3361	3.18×	112	136	1.21×

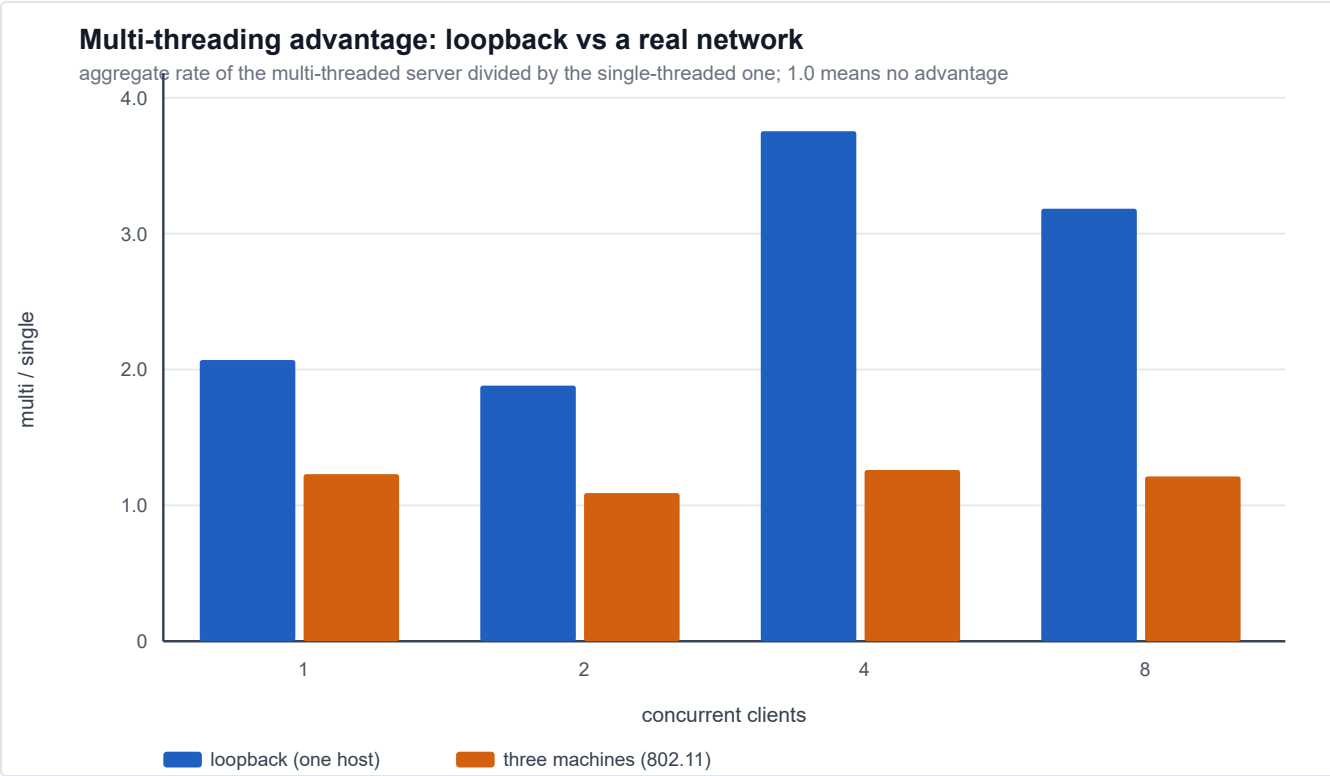


Figure 9. The multi-threaded server's advantage over the single-threaded one, measured on loopback and on three machines. A value of 1.0 would mean no advantage.

> **The headline Part I number depends entirely on where the bottleneck is.** On loopback the multi-threaded server reaches 3.18× the single-threaded aggregate rate at 8 clients. Over a real network the same comparison gives **1.21×**.

Both figures are correct, and they measure different things. Loopback imposes no link limit, so the server's own capacity is the constraint and concurrency in the server is what decides throughput. Over 802.11 the link saturates at 178 Mbit/s and becomes the binding constraint; extra server threads cannot move data a saturated link will not carry, so the architectural advantage shrinks to a steady fifth or so. The multi-threaded server is still the better design at every client count measured — it is simply that the loopback figure describes the server's ceiling, not what a user on a wireless network would experience.

10.3 Parallel connections, measured properly

The Enhanced case is where the two environments disagree most, and where loopback was most misleading.

K	loopback Mbit/s	loopback speedup	network Mbit/s	network speedup
1	196	1.00×	8.9	1.00×
2	225	1.15×	14.9	1.68×
4	366	1.87×	26.2	2.95×
8	337	1.72×	47.2	5.32×

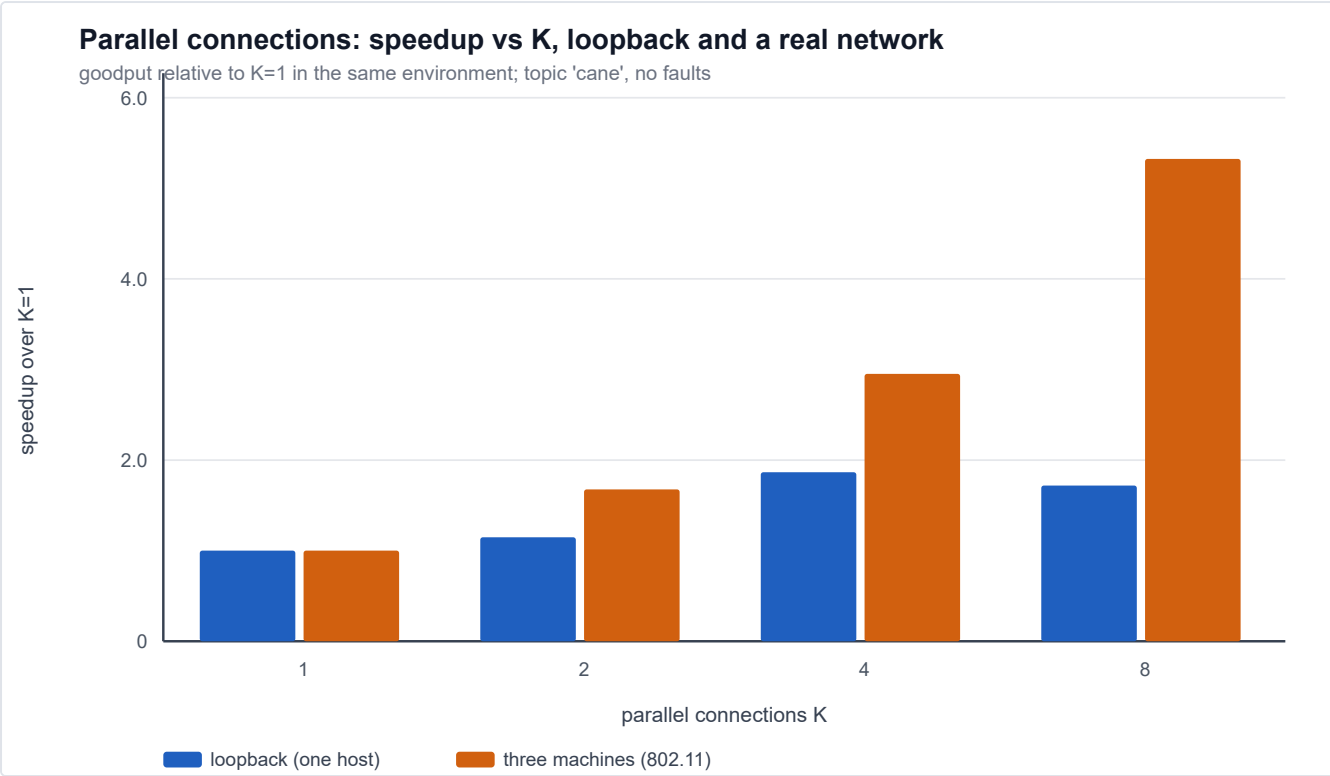


Figure 10. Speedup from K parallel connections relative to K=1 in the same environment, on loopback and on three machines.

> **On loopback, K parallel connections gave 1.87× at best and regressed beyond that. Over a real network, K=8 gives 5.32×, and is still climbing.**

The reason is visible in the absolute numbers. At K=1 the Enhanced client manages only 8.9 Mbit/s over the network — far below what Part I achieves on the same link — because the Enhanced protocol spends a round trip per work item: `REQ_RANGE`, `RESP_RANGE_ACK`, data, `RESP_RANGE_DONE`, and a periodic `REQ_RANGE_ACK`. Topic `cane` holds 4,863 files, so a single-connection transfer serialises tens of thousands of round trips. On loopback a round trip costs microseconds and this is nearly free; on 802.11 it costs milliseconds and dominates the transfer entirely.

That is precisely the cost parallel connections exist to hide. With K connections the client has K round trips in flight at once, and the measured speedup tracks that almost linearly up to K=8. The design rationale in Section 6 — that splitting work across connections escapes a per-flow limit — is sound; loopback simply could not demonstrate it, because loopback has no such limit to escape.

It also identifies a genuine weakness worth stating: the Enhanced protocol's per-work-item handshake is expensive on a latency-bearing link, and a topic of many small files is the worst case for it. Batching several work items into one request, or pipelining the next `REQ_RANGE` before the current range finishes, would attack that directly and is the first change this design should make.

11. Deviations from the design document

`README.md` section 11 lists all of them with reasoning. The ones that affect how this report should be read are:

1. **The resume clamp uses the *sent* position, not the *acknowledged* one.** The design specified `min(client_offset, bytes_acked)`. That is measurably worse: the server's writes land in the socket buffer and run ahead of the acknowledgement stream, so at fault time the acknowledged position trails the client's real progress by roughly a socket buffer rather than one checkpoint interval, and clamping to it re-sent data the client already held. The implementation clamps to the high-water mark of what was actually

transmitted. The acknowledged position is still tracked and reported, because the gap between the two is exactly the acknowledgement lag.

2. **Every Enhanced transfer is a range request.** The design had a separate whole-file path for small files; the implementation makes a small file a one-item range, so the server has one state machine instead of two. Six message types beyond the planned pair were needed to make *K* connections work.
3. **Part I is one** `server.c` with a mode argument, rather than `server_single.c` plus `server_multi.c`: the two modes share the whole request path.
4. **The main matrix is single-host; the required three-machine experiment is reported separately.** The design document planned every measurement on three machines. In practice the full seven-experiment matrix was run on one host over loopback, because it needs several hours and over 20 GB of transfer, and Part I plus the Enhanced K sweep were then repeated on three machines (Section 10). That was the right split: the loopback matrix supplies the byte accounting, which is link-independent, and the three-machine runs supply the throughput figures, which are not. Running only the former would have left two conclusions wrong, as Section 10 shows.

12. Correctness and verification

Performance numbers are only meaningful if the transfers were correct, so correctness is established independently of the benchmarks.

- **366 automated checks pass** — 120 unit checks over `common/`, plus four functional suites (57 for Part I, 43 for Case 1, 56 for Case 2, 90 for Enhanced). Between them they cover the empty file and the chunk-size boundaries, concurrent clients against both threading modes, rejection of malformed and unsafe names, truncated and illegal messages, every error code, fault injection and resume, checkpoint correctness across a client kill, and byte-for-byte comparison of the received tree by both `diff -r` and an aggregate SHA-256.
- **48 sanitizer checks pass** — every part under AddressSanitizer with UndefinedBehaviorSanitizer, and under ThreadSanitizer, with no report.
- **Every benchmark run is itself a correctness test.** Each client verifies the CRC-32 of every file it writes. Across the loopback campaign 2,147,270 files were verified with **0** mismatches, and across the three-machine campaign a further 787,806 files with **0** mismatches — the latter over a real wireless link, between a Linux server and two macOS clients.
- **The implementation is portable.** The same source built and ran correctly on both Linux and macOS. One genuine portability defect was found and fixed in the process: `common/protocol.c` passed `MSG_NOSIGNAL` to `send()`, which is Linux-only, so the client would not compile on macOS at all. It now falls back to `0` where the platform lacks the flag, which is correct because every program here already installs `SIG_IGN` for `SIGPIPE`.
- **Byte-identical output against the real dataset** was confirmed separately with `diff -r` and an aggregate SHA-256 over the received tree.

13. Summary of observations

1. **Multi-threading wins, but by how much depends on the bottleneck.** At 8 clients the multi-threaded server delivers 3.18× the single-threaded aggregate rate on loopback, where the server is the constraint, and 1.21× over a real network, where the link saturates and limits both. It is the better design in both, but only the second figure describes what a user would see.
2. **Session management converts an unbounded cost into a bounded one.** Case 1's redundant traffic grows as $(1-p)^{-C}$ and is therefore unbounded in both fault rate and topic size; Case 2's is bounded by one checkpoint interval per fault. At p=0.002 that is a 60395× difference in bytes retransmitted.

3. **Fault tolerance is not free when there are no faults.** Case 2 pays roughly 6.4× in fault-free throughput on this test bed for checkpoint persistence and acknowledgement traffic. The break-even fault rate is nonetheless very low.
4. **Parallel connections are the single most valuable enhancement — but only on a real link.** On loopback they gave at best 1.87× and regressed past K=4. Over 802.11, K=8 gives **5.32×** and has not yet plateaued. The Enhanced protocol spends a round trip per work item, which is free on loopback and dominant on a real link, and K connections hide exactly that. Judged on loopback alone the feature looks barely worth its complexity; judged properly it is the biggest win in the project.
5. **A real network reverses conclusions, it does not merely scale them.** Two of this report's findings — the size of the threading advantage and the value of parallel connections — moved in opposite directions when measured on three machines rather than one. Byte counts were unaffected throughout. That is the strongest methodological lesson here: a loopback benchmark measures the implementation, not the system.
6. **Protocol framing overhead is negligible throughout** — under 1 % in every configuration, and lower for topics with larger files. At high fault rates Case 1's overhead is dominated by re-sent payload, not by headers.
7. **Topic shape matters as much as topic size.** Mean file size drives both overhead and achievable rate, because per-file costs are amortised over more payload.

14. Limitations and future work

- **The fault-injection experiments were run only on loopback.** Part I and the Enhanced K sweep were repeated on three machines; Case 1 against Case 2 was not. Their headline result is a byte count, which is link-independent, so the conclusion stands — but the *time* penalty of Case 1 is understated here. Section 5 reports Case 1 moving 471.8 MB of redundant data at p=0.002 while finishing only 15.6× slower, because re-sending costs almost nothing on loopback. At the 178 Mbit/s measured over 802.11 those same redundant bytes would cost roughly half a minute of real time, so session management is worth considerably more on a real link than these figures suggest. Repeating E3 on the three-machine bed is the obvious next measurement.
- **K was not swept past 8.** The network speedup was still rising at K=8, so the optimum was not located. The protocol caps `MAX_CONNS_PER_SESSION` at 16; measuring 12 and 16 would find where per-connection cost finally overtakes the latency hiding.
- **One wireless link, one topology.** The three-machine results come from a single 802.11 hotspot with two macOS clients. A wired link, or a link with materially different latency, would test whether the round-trip sensitivity identified in Section 10.3 scales the way the explanation predicts.
- **Server session state is in memory only.** If the server restarts mid-experiment, resumes against it are rejected and clients fall back to a fresh session — correct, but slower. Persisting the session table would remove that case.
- **The checkpoint interval was not swept.** 64 KB is exposed as `--ckpt`; given that checkpointing is the measured cost of Case 2, a sweep would locate the real trade-off between fault-free throughput and worst-case redundant bytes.
- **Fault injection is server-side only.** Client-side crashes are exercised by the Case 2 test suite, which kills a client mid-transfer and resumes it, but not by the injection mechanism itself.
- **Thread-per-client does not bound thread count.** Appropriate for the client counts used here; a thread pool would be the next step for deployment beyond the lab.